EXERCICE 11 · CODE COMPLET

Page Comments avec un custom hook

Tous les fichiers utilisés, en entier, dans l'ordre de lecture.

1. **correction** (4 fichiers)

1 correction

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (74 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg, package.json

FICHIER **correction/src/main.jsx** — 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

FICHIER **correction/src/App.jsx** — 12 lignes

```jsx
import "./App.css";
import Comments from "./components/Comments";

function App() {
  return (
    <main className="page">
      <Comments />
    </main>
  );
}

export default App;
```

FICHIER **correction/src/components/Comments.jsx** — 41 lignes

```jsx
import useFetchApi from "../hooks/useFetchApi";

function Comments() {
  // Toute la logique de fetch est dans le custom hook useFetchApi : la page ne garde que l'affichage
  const {
    data: comments,
    isLoading,
    isError,
  } = useFetchApi("https://jsonplaceholder.typicode.com/comments?postId=1");

  return (
    <section className="card">
      <h2 className="title">Comments</h2>
      {isError && <p className="error">Erreur de chargement</p>}
      {isLoading ? (
        <p className="loading">Loading ...</p>
      ) : (
        <table className="table">
          <thead>
            <tr>
              <th>Id</th>
```

```
              <th>Comment</th>
              <th>Email</th>
            </tr>
          </thead>
          <tbody>
            {comments.map((comment) => (
              <tr key={comment.id}>
                <td>{comment.id}</td>
                <td>{comment.name}</td>
                <td>{comment.email}</td>
              </tr>
            ))}
          </tbody>
        </table>
      )}
    </section>
  );
}

export default Comments;
```

FICHIER `correction/src/hooks/useFetchApi.js` — 33 lignes

```
import { useEffect, useState } from "react";

// Custom hook useFetchApi : la logique de fetch (data, loading, error), réutilisable : seule l'URL change
function useFetchApi(url) {
  const [data, setData] = useState([]);
  const [isLoading, setIsLoading] = useState(false);
  const [isError, setIsError] = useState(false);

  // useEffect avec [url] : le fetch se relance si l'URL change
  useEffect(() => {
    const getData = async () => {
      setIsLoading(true);
      try {
        const response = await fetch(url);
        // on vérifie la réponse avant de lire le JSON
        if (!response.ok) {
          throw new Error("Network response was not ok");
        }
        const json = await response.json();
        setData(json);
      } catch {
        setIsError(true);
      } finally {
        setIsLoading(false);
      }
    };
    getData();
  }, [url]);

  return { data, isLoading, isError };
}

export default useFetchApi;
```